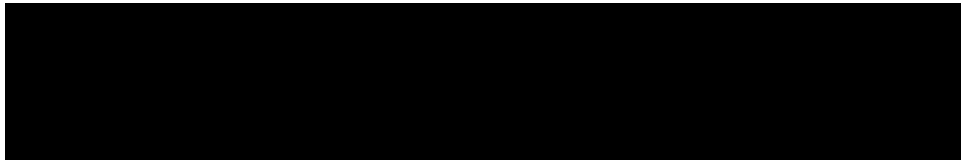


# Fractality of the Meta-loss Landscape: A Replication Study



## 1 Abstract

This paper replicates and further investigates the findings of Sohl-Dickstein et al. on the fractal nature of neural network trainability. We explore the emergence of fractal patterns in the hyperparameter landscapes of neural networks during training, focusing on the boundary between convergent and divergent training dynamics. By systematically varying hyperparameters and visualizing the resulting training outcomes, we observe complex, scale-invariant structures in the trainability boundary across multiple experimental conditions, including different network architectures, activation functions, and training methodologies. Our findings confirm the presence of fractal geometry in the hyperparameter landscape and highlight the sensitivity of neural network training to subtle variations in hyperparameters. We discuss the implications of these fractal structures for hyperparameter optimization, model interpretability, and generalization. This work contributes to a deeper understanding of the complex dynamics underlying neural network training and suggests potential avenues for developing more robust and efficient training algorithms that can navigate the fractal landscape of hyperparameters.

## 2 Introduction

Artificial neural networks have proven to be powerful tools for solving complex problems across various domains [1]. However, training these models effectively is challenging due to the high-dimensional and non-convex nature of the optimization landscape [2, 3]. Hyperparameters, such as learning rates and network architecture, play a crucial role in determining the performance of neural networks [4]. Tuning these hyperparameters is essential but time-consuming and computationally expensive [5]. Meta-learning aims to automate hyperparameter optimization by learning to learn [6, 7]. It involves refining the learning algorithm to focus on the automated optimization of hyperparameters, reducing the need for manual tuning and increasing model performance [8].

Recent research has revealed a connection between the trainability of neural networks and fractal geometry [9]. Fractals are complex patterns that exhibit self-similarity and scale invariance [10, 11]. In neural networks, fractal geometry has been observed in the boundary between hyperparameters that lead to convergent and divergent training dynamics [12]. This boundary, known as the edge of stability, is a precarious region in the hyperparameter space that delineates regions leading to convergent (successful training) and divergent (unsuccessful training) behaviors [14]. Optimal training configurations are often found adjacent to this boundary, where the neural network is sensitive enough to learn effectively without diverging [15].

The meta-loss landscape, which depicts the performance of neural networks against hyperparameter variations, is inherently chaotic [3]. Minor hyperparameter changes can lead to disproportionate impacts on neural network training outcomes, reflecting high sensitivity to hyperparameter adjustments and complicating optimization efforts [16]. Sohl-Dickstein et al. [12] investigated the relationship between hyperparameters and neural network trainability, focusing on the boundary separating successful and unsuccessful training regimes. They discovered that this boundary exhibits fractal properties, with complex patterns recurring across multiple scales, suggesting that trainability is a complex phenomenon governed by fractal geometry.

The fractal nature of the trainability boundary has implications for hyperparameter optimization and meta-learning. Traditional approaches to hyperparameter tuning may struggle to navigate the fractal landscape efficiently [4, 5]. Moreover, small perturbations in hyperparameters can lead to drastic changes in training outcomes, making the optimization process highly sensitive and potentially unstable.

Understanding the fractal properties of neural network trainability is crucial for developing more effective and efficient training algorithms. By leveraging insights from fractal geometry, researchers can design meta-learning techniques better suited to navigate the complex hyperparameter landscape [7, 8].

In this paper, we aim to replicate and further investigate the findings of Sohl-Dickstein et al. [13] on the fractal nature of neural network trainability. We conduct experiments on different neural network architectures and datasets to validate the presence of fractal boundaries in the hyperparameter landscape and explore their implications for meta-learning and hyperparameter optimization. Our goal is to contribute to the understanding of the complex dynamics underlying neural network training and provide insights that can guide the development of more effective and efficient learning algorithms.

### 3 Methodological Framework

Our methodology combines the principles of fractal geometry with the practical aspects of neural network optimization to analyze the relationship between

hyperparameters and training outcomes.

[\(Link to the video of fractal zoom\)](#)

### 3.1 Neural Network Architecture and Initialization

We consider a one-hidden-layer neural network with input dimension  $n$  and two sets of weights:  $W_0 \in R^{n \times n}$  (input to hidden) and  $W_1 \in R^{1 \times n}$  (hidden to output). The network is trained on a mean squared error (MSE) loss for a regression task, aiming to predict a continuous output ( $target\_dim = 1$ ) based on the input features. The weights and biases of the network are initialized from a standard normal distribution  $\mathcal{N}(0, 1)$  to break symmetry and ensure diverse initial activations. The hidden layer uses a nonlinear activation function  $\sigma(\cdot)$ , which can be either the hyperbolic tangent ( $\tanh$ ) or the rectified linear unit (ReLU).

### 3.2 Training Dataset Generation

To capture the dynamics of neural network training, we generate a synthetic dataset  $\mathcal{D}$  consisting of input features  $X \in R^n$  and corresponding target values  $Y \in R^d$ , both sampled from a standard normal distribution  $\mathcal{N}(0, 1)$ . The size of the dataset  $|\mathcal{D}|$  is determined by the number of free parameters in the network, ensuring a balanced representation of the model’s complexity. For non-linear networks with a  $\tanh$  or ReLU activation function, the dataset size is set to be proportional to the total number of free parameters, calculated as  $|\mathcal{D}| = \alpha n_p / d$ , where  $\alpha$  is a proportionality constant and  $d$  is the dimension of the output. In the case of deep linear networks (i.e., networks with an identity activation function), the dataset size is determined as  $|\mathcal{D}| = \alpha n$ , where  $n$  is the width of the layers. By carefully controlling the dataset size, initialization, and hyperparameter grid, the data generation and preparation process ensures a systematic exploration of the hyperparameter landscape. This approach enables the analysis of fractal patterns in the training dynamics while balancing computational efficiency and representational adequacy.

### 3.3 Hyperparameter Grid Search

To explore the hyperparameter landscape, we perform a grid search over the learning rates for the input and output layers, denoted as  $\eta_0$  and  $\eta_1$ , respectively. The learning rates are varied logarithmically over a specified range of  $[-3, 6]$  for both  $\eta_0$  and  $\eta_1$ , creating a 2D grid of hyperparameter combinations.

For each combination of learning rates  $(\eta_0, \eta_1)$  in the grid, we train the neural network using the generated dataset  $\mathcal{D}$ . The training process involves iterative updates of the network’s parameters  $\theta$  using full-batch gradient descent optimization:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t, X, Y), \quad (1)$$

where  $\mathcal{L}(\theta, X, Y)$  is the MSE loss function, and  $\eta$  is the learning rate. The number of training iterations was varied between  $10^2$  and  $10^3$ .

Each pixel in the resulting hyperparameter landscape visualization corresponds to a single training run with a specific combination of learning rates  $(\eta_0, \eta_1)$ . The color of each pixel represents the convergence behavior of the network for that particular hyperparameter configuration.

### 3.4 Convergence Measure and Visualization

To analyze the training dynamics, we define a convergence measure that quantifies the stability and speed of convergence for each hyperparameter combination. The convergence measure is based on the loss trajectory during training, considering factors such as the final loss value, the rate of convergence, and the presence of oscillations.

In the case of the loss readout, the convergence measure is calculated as follows:

- The loss values are normalized by dividing them by the initial loss value.
- If the normalized loss exceeds a maximum threshold, it is clipped to the threshold value.
- Convergence is determined by checking if the average of the last 20 normalized loss values is less than 1.
- If convergence is achieved, the negative sum of the normalized loss values is returned as the convergence measure.
- If convergence is not achieved, the sum of the reciprocal of the normalized loss values is returned.

The training outcomes are categorized based on the convergence or divergence of the loss function. Convergent runs, where the loss decreases and stabilizes, are indicated by a binary function  $\mathcal{C}(\eta_0, \eta_1) = 1$ . Divergent runs, characterized by increasing or oscillating loss values, are indicated by  $\mathcal{C}(\eta_0, \eta_1) = 0$ . The boundary between convergent and divergent regions in the hyperparameter space is of particular interest for studying fractal patterns.

We visualize the hyperparameter landscape using a color-coded scheme, where each pixel in a 2D grid represents a specific combination of learning rates  $(\eta_0, \eta_1)$ . The color of each pixel indicates the convergence measure, with blue representing stable and fast convergence, and red representing divergence.

or slow convergence. The intensity of the colors reflects the magnitude of the convergence measure.

### 3.5 Fractal Boundary Analysis

To investigate the fractal nature of the trainability boundary, we focus on the regions where the convergence measure transitions from stable to unstable. We apply zooming techniques to magnify these boundary regions, revealing self-similar patterns and structures reminiscent of fractal geometry. The fractal dimension of the trainability boundary was estimated using the box-counting method, implemented in the porespy library [16]. The edges of the convergence measure images were extracted, and the boxcount method was applied to count the boxes containing the boundary at varying scales. The fractal dimension, estimated from the slope of the linear regression line fitted to the log-log relationship between box size and box count, quantifies the complexity of the boundary, with values closer to 2 indicating a more fractal and space-filling structure. The median fractal dimension estimate across the zoom sequence was reported for each experimental condition.

The zooming process is performed by generating a sequence of zoomed-in images centered around a specific point in the hyperparameter space. The number of zoom levels and the zoom factor can be specified to control the magnification of the boundary regions. To ensure high-quality visualizations, we increase the resolution of the zoomed-in images. This function generates new images with a higher resolution for each zoom level, allowing for a more detailed examination of the fractal patterns.

### 3.6 Comparative Experiments

To assess the generality of the fractal patterns, we conduct a series of comparative experiments by varying the neural network architecture, activation functions, and training methodologies. We investigate the impact of these factors on the emergence and characteristics of the fractal boundaries.

Specifically, we explore the following variations:

- 1.** Activation functions: tanh, ReLU, and identity (linear);
- 2.** Training modes: full-batch gradient descent and mini-batch stochastic gradient descent;
- 3.** Dataset size: single training point and larger datasets;
- 4.** Hyperparameter subspaces: learning rates and weight initialization scales.

By systematically varying these experimental conditions, we aim to gain insights into the universality and adaptability of the fractal patterns in neural network trainability.

### 3.7 Computational Considerations

The exploration of the hyperparameter landscape and the generation of fractal visualizations are computationally expensive tasks. Each pixel in the visualization requires a separate training run of the neural network, and the grid search involves a large number of hyperparameter combinations.

To handle the computational demands, we utilize an NVIDIA A100 GPU, which provides high-performance computing capabilities. The A100 GPU enables parallel processing of multiple training runs, significantly reducing the overall computation time. However, despite the use of advanced hardware, the computational cost remains substantial. The number of training runs grows exponentially with the resolution of the hyperparameter grid and the number of experimental variations. Therefore, we carefully design our experiments to balance computational feasibility with the desired level of detail and analysis.

The experiments are implemented using the JAX library in Python, which provides efficient numerical computing and automatic differentiation capabilities. The code for generating the datasets, training the neural networks, and visualizing the hyperparameter landscapes is provided in the accompanying repository.

## 4 Results

We systematically investigate the emergence of fractal patterns in the trainability landscape of neural networks by varying key factors such as activation functions, dataset sizes, and training methodologies. Our results demonstrate the robustness and universality of these fractal structures across different experimental configurations.

We first examine the impact of the hyperbolic tangent ( $\tanh$ ) activation function on the fractal geometry of the trainability landscape. By setting the dataset size to be half the number of free parameters in the network and training the model using both full-batch and mini-batch gradient descent (batch size = 32), we observe striking fractal patterns in the resulting hyperparameter landscapes. The boundary between convergent and divergent regions exhibits self-similarity across multiple scales, indicating the presence of a fractal structure, with a fractal dimension of 1.68.

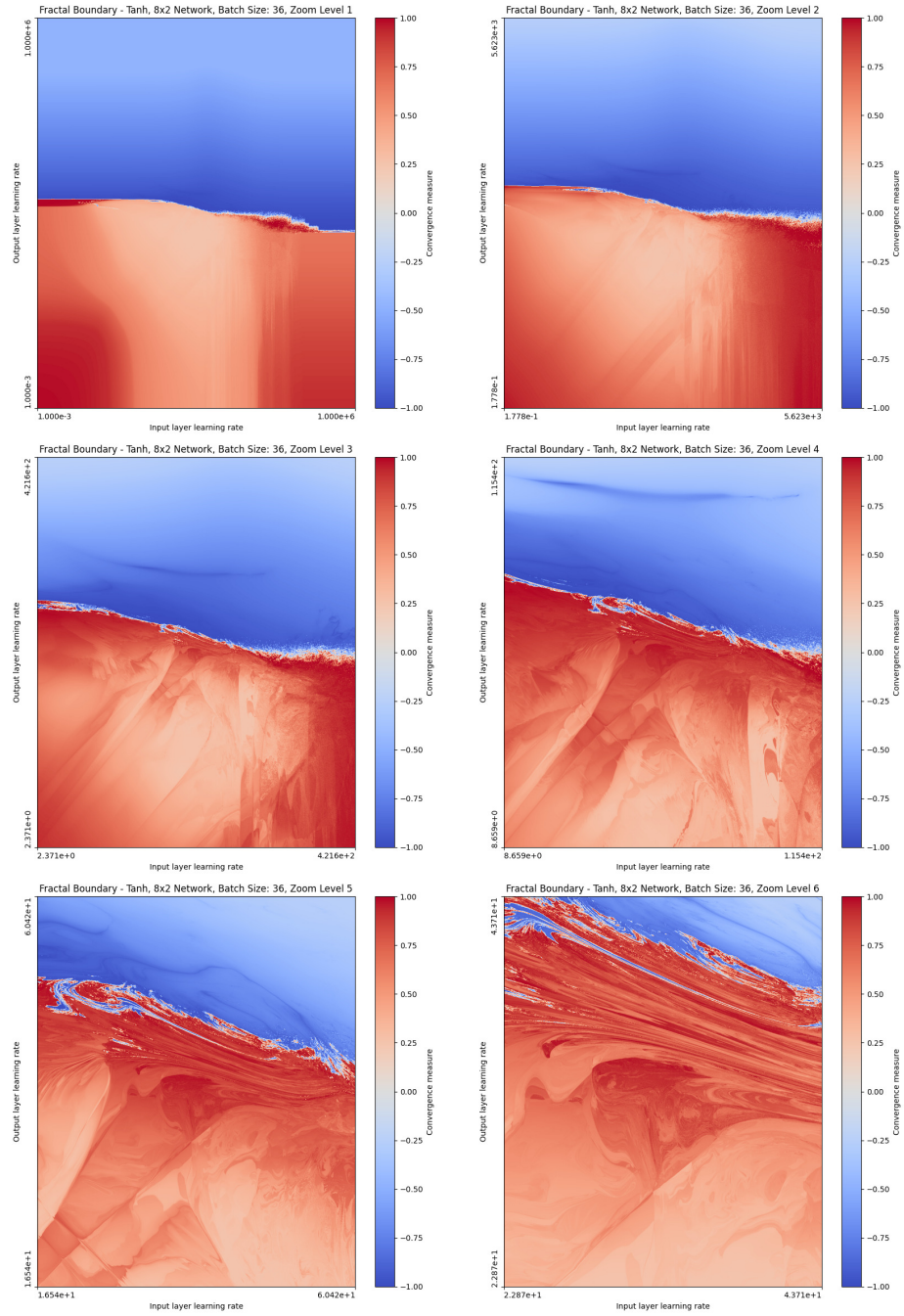


Figure 1: Fractal boundary between trainable (blue) and untrainable (red) hyperparameters. Paler points indicate faster convergence or divergence.

The resulting visualizations reveal qualitatively similar fractal patterns, suggesting that the emergence of fractal boundaries is not specific to a particular activation function. However, the choice of activation function does appear to influence the shape and density of the fractal structures, highlighting the complex interactions between model architecture and trainability.

By increasing the dataset size to be equal to the number of free parameters in the network, we explore how the fractal patterns evolve as the model’s capacity to learn from the data changes. The persistence of fractal structures in these experiments confirms the robustness of the phenomenon across different dataset sizes.

| Experimental Condition                      | Fractal Dimension |
|---------------------------------------------|-------------------|
| Tanh, full batch, D(0.5)                    | 1.68              |
| ReLU, full batch, D(0.5)                    | 1.30              |
| Tanh, full batch, D(1)                      | 1.66              |
| ReLU, full batch, D(1)                      | 1.28              |
| Identity (linear), full batch, D(1)         | 1.16              |
| Tanh, minibatch, D(1)                       | 1.55              |
| Tanh, single datapoint, D(1)                | 1.31              |
| Tanh, initialization vs learning rate, D(1) | 1.95              |

Table 1: Fractal dimension estimations for different experimental conditions. D stands for the ratio between the dataset size and the number of parameters in the network

The fractal dimension estimates in Table 1 reveal the presence of intricate, self-similar patterns in the trainability boundaries of neural networks across various experimental conditions. Comparing the fractal dimensions for different activation functions reveals that the choice of activation function influences the complexity of the trainability boundary. The tanh activation function consistently yields higher fractal dimensions compared to the ReLU activation function, suggesting that the tanh function induces more elaborate and detailed fractal structures. This observation holds true for both D(0.5) and D(1) dataset sizes. Interestingly, the identity (linear) activation function results in the lowest fractal dimension of 1.16, indicating a relatively simpler and smoother trainability boundary. This finding highlights the role of nonlinearity in shaping the fractal geometry of the hyperparameter landscape.

The fractal dimensions remain relatively stable across different dataset sizes for the tanh activation function, demonstrating the robustness of the fractal patterns. However, minibatch training leads to a slightly lower fractal dimension compared to full-batch training, suggesting that the stochastic nature of minibatch training may result in a marginally smoother trainability boundary. Remarkably, the highest fractal dimension of 1.95 is observed when exploring



the relationship between initialization and learning rate, indicating the presence of highly fractal structures in this hyperparameter subspace. This finding underscores the importance of considering interactions between initialization and learning rate in shaping the trainability landscape.

## 5 Conclusion

Despite the computational limitations imposed by the intensive nature of the experiments, the fractality of the trainability landscape emerges clearly in all the explored configurations. Even with a modest zoom factor of 10x, the fractality is discernible. Higher magnifications could potentially reveal additional layers of structure, but the visualization and interpretation of the fractal boundary become increasingly challenging in high-dimensional hyperparameter spaces.

The fractal nature of the trainability landscape has significant implications for understanding and optimizing neural network training. The scale-invariant properties of the fractal boundaries suggest that the principles governing convergence and divergence operate across a wide range of hyperparameter values. This insight could guide the development of more efficient hyperparameter optimization techniques that exploit the fractal structure to navigate the search space effectively.

Moreover, the fractal patterns in the trainability landscape raise important questions about the nature of generalization in neural networks. The fractal boundary between convergent and divergent regions may hold clues to the model’s ability to learn meaningful representations and avoid overfitting. Investigating the relationship between fractal geometry and generalization could provide new perspectives on the design of robust and reliable learning algorithms.

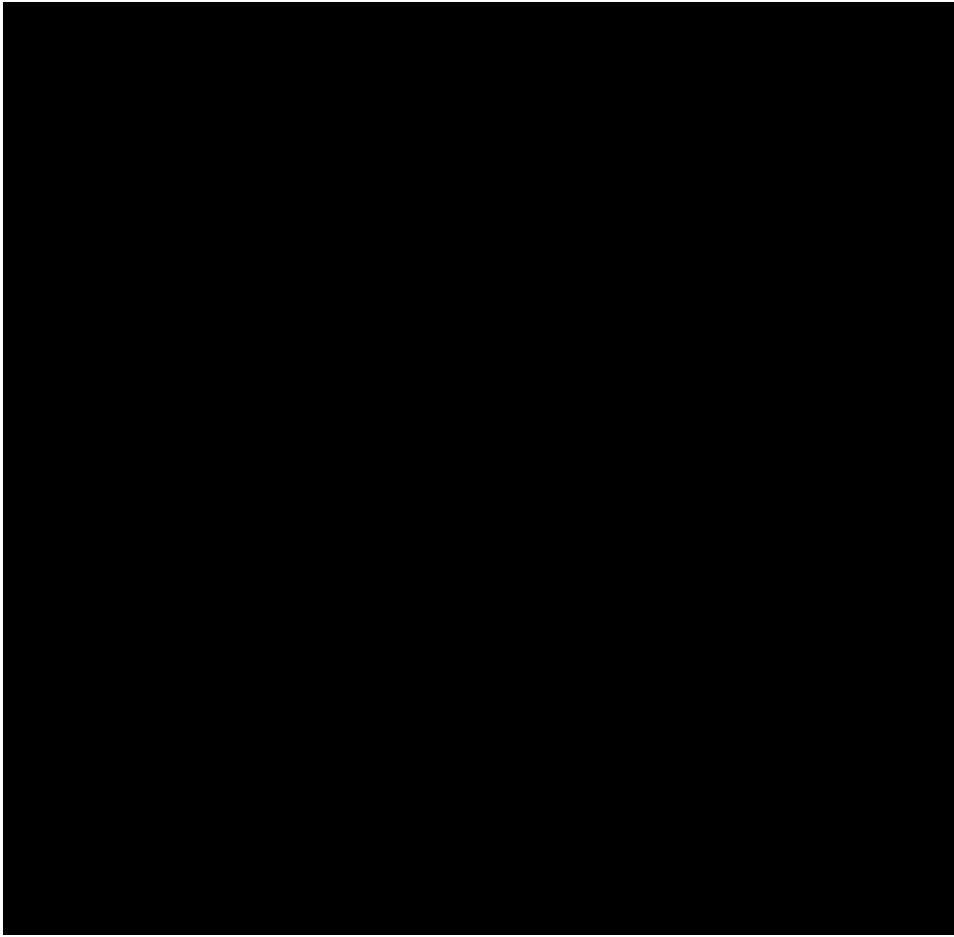
Future research directions include the development of advanced visualization methods, such as adaptive zooming algorithms, to provide a more detailed view of the fractal boundaries. Quantitative analysis techniques, such as estimating the fractal dimension and characterizing self-similarity across scales, could yield further insights into the complexity and universality of the observed patterns.

In summary, our results show fractal geometry that emerges in the trainability landscape of neural networks. The robustness and universality of these fractal patterns across different experimental configurations highlight the fundamental nature of this phenomenon.

## 6 References

1. LeCun, Y., Bengio, Y., Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

2. Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep learning. MIT Press.
3. Li, H., Xu, Z., Taylor, G., Studer, C., Goldstein, T. (2018). Visualizing the loss landscape of neural nets. Advances in Neural Information Processing Systems, 31. arXiv:1712.09913
4. Bergstra, J., Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13(2).
5. Snoek, J., Larochelle, H., Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. Advances in Neural Information Processing Systems, 25. arXiv:1206.2944
6. Vanschoren, J. (2018). Meta-learning: A survey. arXiv:1810.03548.
7. Finn, C., Abbeel, P., Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In International Conference on Machine Learning (pp. 1126-1135). PMLR. arXiv:1703.03400
8. Hospedales, T., Antoniou, A., Micaelli, P., Storkey, A. (2020). Meta-learning in neural networks: A survey. arXiv preprint arXiv:2004.05439.
9. Dym, N., Sober, B., Daubechies, I. (2020). Expression of fractals through neural network functions. IEEE Journal on Selected Areas in Information Theory, 1(1), 57-66. arXiv:1905.11345
10. Mandelbrot, B. B. (1983). The fractal geometry of nature. W. H. Freeman.
11. Iannaccone, P. M., Khokha, M. (Eds.). (1996). Fractal geometry in biological systems: An analytical approach. CRC Press.
12. Sohl-Dickstein, J. (2022). The boundary of neural network trainability is fractal. arXiv preprint arXiv:2402.06184.
13. Poole, B., Lahiri, S., Raghu, M., Sohl-Dickstein, J., Ganguli, S. (2016). Exponential expressivity in deep neural networks through transient chaos. Advances in Neural Information Processing Systems, 29. arXiv:1606.05340
14. Schoenholz, S. S., Gilmer, J., Ganguli, S., Sohl-Dickstein, J. (2017). Deep information propagation. arXiv preprint arXiv:1611.01232.
15. Novak, R., Bahri, Y., Abolafia, D. A., Pennington, J., Sohl-Dickstein, J. (2018). Sensitivity and generalization in neural networks: An empirical study. arXiv preprint arXiv:1802.08760.
16. Gostick et al., "PoreSpy: A Python Toolkit for Quantitative Analysis of Porous Media Images," Journal of Open Source Software, 4(37), 1296, 2019. DOI: 10.21105/joss.01296



## Appendix

The appendix contains three sets of figures. The first set shows the fractal boundary between trainable and untrainable hyperparameters for the Tanh activation function with full-batch training. The second set shows the fractal boundary for the Tanh activation function when exploring the relationship between initialization and learning rate. The third one shows the fractal boundary for ReLU, full-batch.

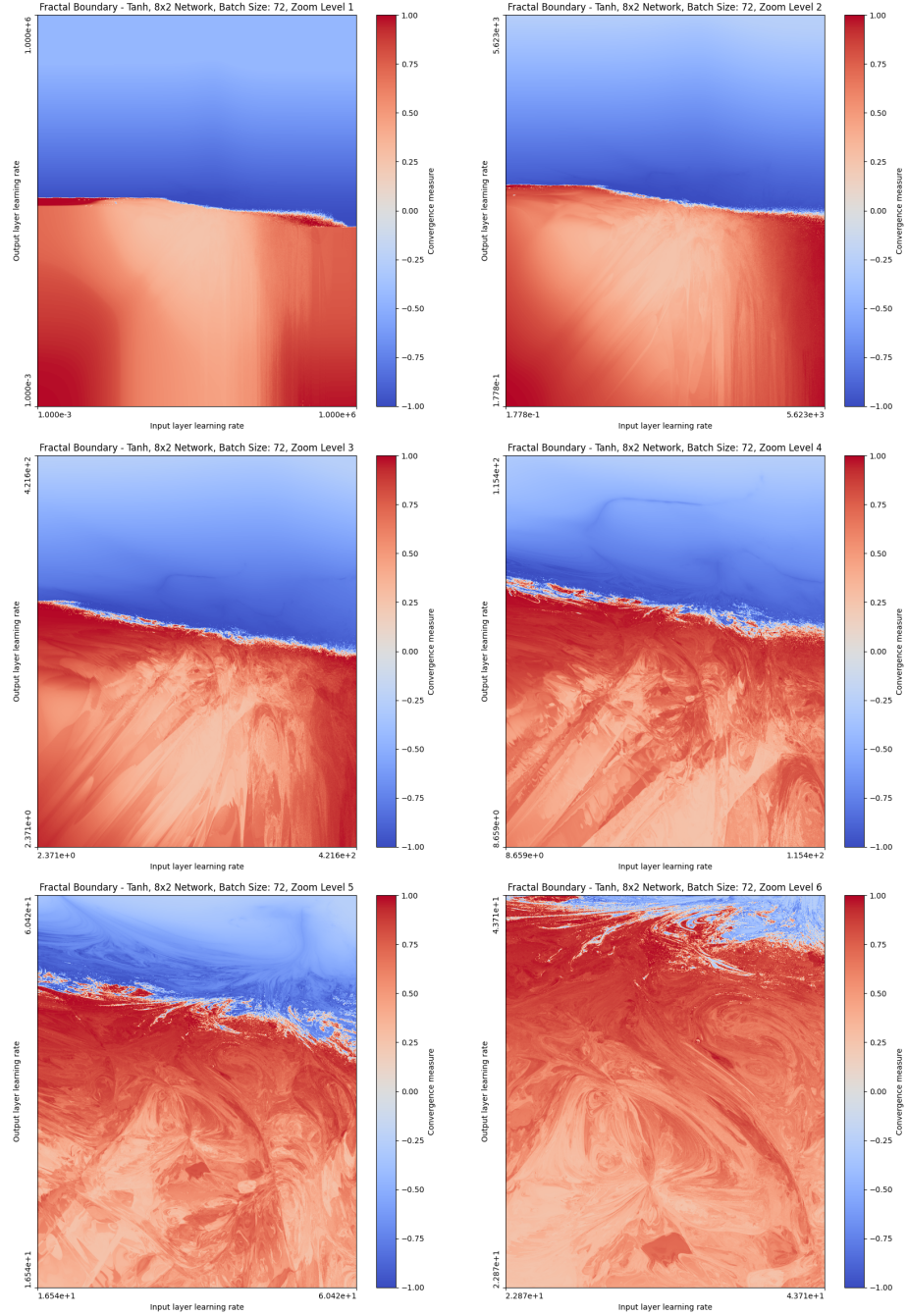


Figure 2: Tahn, full batch.

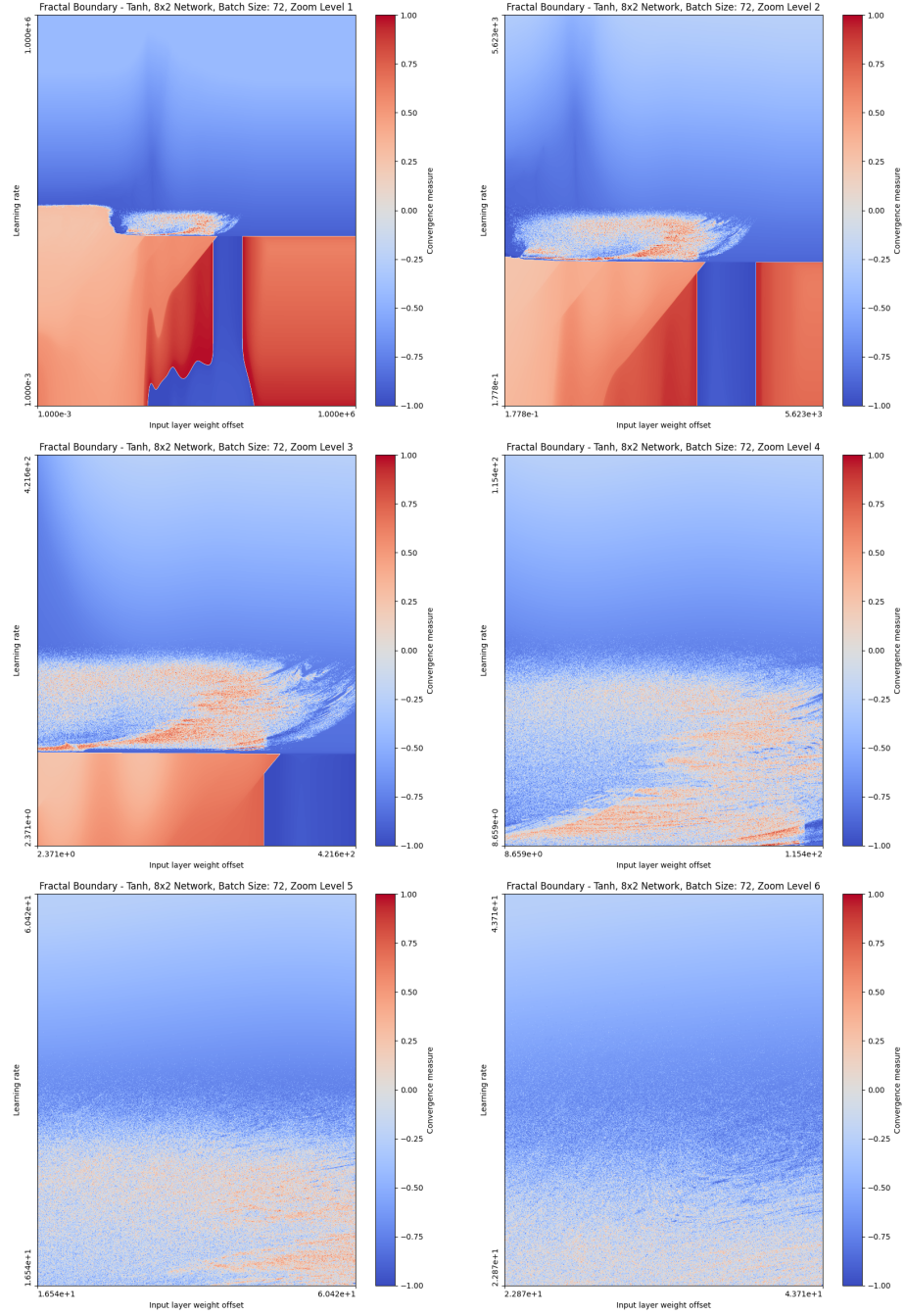


Figure 3: Tanh, initialization vs learning rate, which has the highest fractal dimension (1.95)

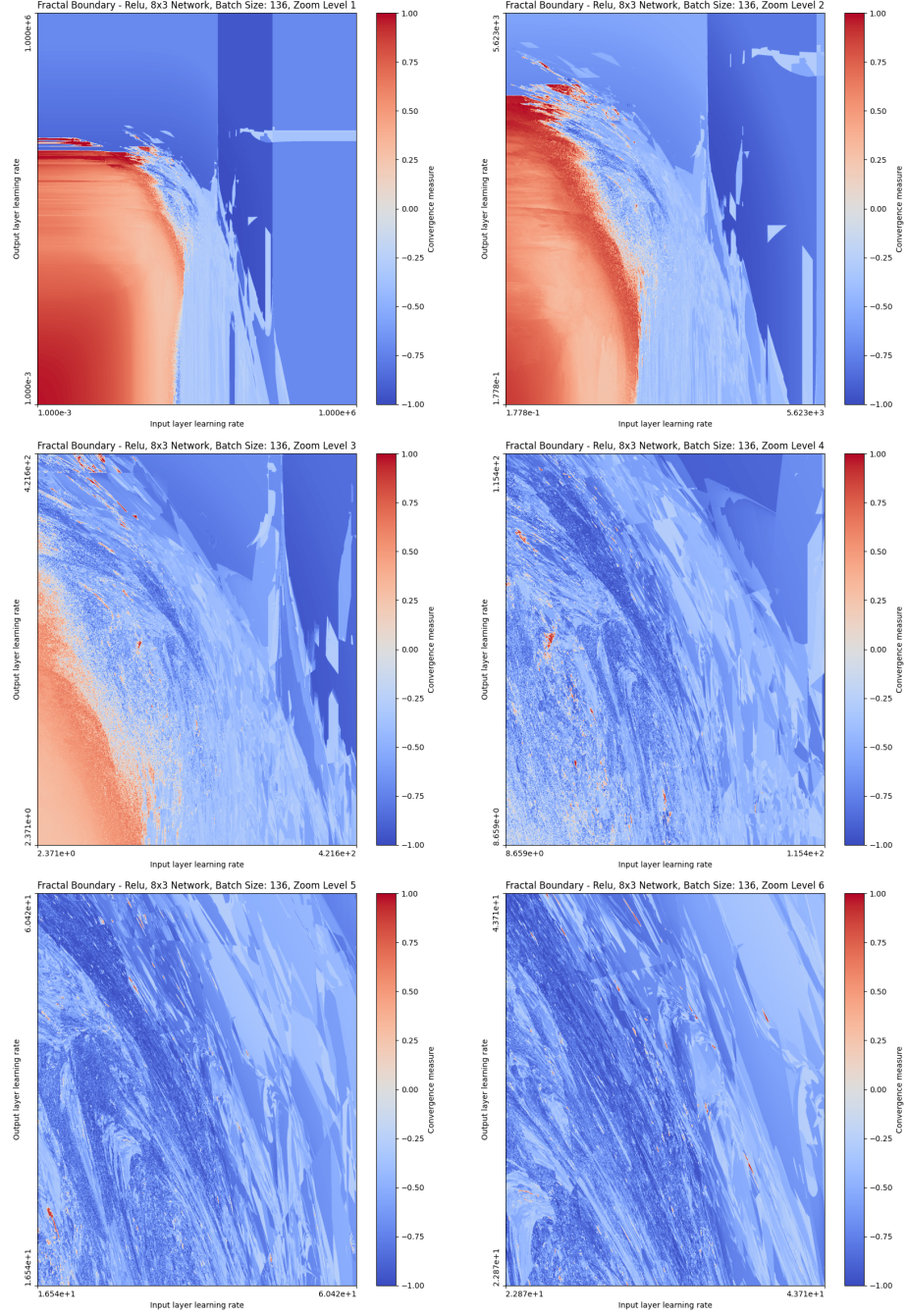


Figure 4: ReLU, full batch